Appendix. Use Cases: Industry Use Cases

This appendix includes prompt templates for various industry use cases.

A prompt template is a structured framework for guiding Solar's responses. By using defined instructions and formats, you can generate ideas for repetitive tasks and improve efficiency through template reuse.

Table of Contents

- Use `Ctrl + F` (Windows) or `Cmd + F` (Mac) to locate specific sections by title.
- **A.1 Document Formatting**
- **A.2 Entity Extraction**
  - A.2.1 Function Calling: Entity Extraction
- **A.3 Remove PII (Personally Identifiable Information)**
- **A.4 Complex Prompt: Query Decomposition**

Set up

```
from openai import OpenAI

# Retrieve the UPSTAGE_API_KEY variable from the IPython store
%store -r UPSTAGE_API_KEY

try:
    if UPSTAGE_API_KEY:
        print("Success!")
except NameError as ne:
    print(f"Since, {ne}")
    print("Please, insert your API key.")
    UPSTAGE_API_KEY = input("UPSTAGE_API_KEY =")

# Set your API key:
# UPSTAGE_API_KEY = " " <- Insert your API key here.

client = OpenAI(
    api_key= UPSTAGE_API_KEY,
    base_url="https://api.upstage.ai/v1/solar"
)

config_model = {
    "model": "solar-pro",
    "max_tokens": 996,
    "temperature": 0.5,
    "top_p": 1.0,
```

```
}

def get_completion(messages, system_prompt="", config=config_model):
    try:
        if system_prompt:
            messages = [{"role": "system", "content": system_prompt}]
+ messages

        message = client.chat.completions.create(messages=messages,
**config)
        return message.choices[0].message.content

    except Exception as e:
        print(f"Error during API call: {e}")
        return None

Success!
```

A.1 Document Formatting

Ensuring consistency in references and citations is crucial for readability and professionalism. When working with texts that contain citations in various formats (e.g., author-date, footnotes, endnotes), automating the process of identifying and reformatting them can save time and reduce errors.

The goal is to:

- Identify all citations and references in the text, regardless of their existing format.
- Reformat them to a consistent style using sequential numeric indices enclosed in square brackets (e.g., [1], [2], [3]).
- Maintain sequential numbering throughout the document to reflect the order in which the citations appear.
- Preserve the rest of the text without alterations.

By crafitng a precise prompt, you can guide Solar to perform this task efficiently.

Prompt Template

```
message = [
    {
        "role": "user",
        "content": """Your task is to find all references and
citations in the provided text and reformat them into a consistent
numeric citation style.
```

```
#Instructions
1. Identify all citations and references in the given text.
2. Replace each citation with a sequential numeric index enclosed in
square brackets (e.g., [1], [2], [3]), following the order they appear
in the text.Follow the Chicago Manual of Style convention.
3. Update any in-text citations to this format.
4. Do not modify any other content in the text.
--
<Text>
A substantial amount of research has investigated the rhetoric and
argumenttechniques employed in political debates, with a particular
focus on televiseddebates (Benoit & Wells, 1996; Benoit & Brazeal,
2002; Clayman & Heritage,2002a, 2002b). This body of work often
employs content analysis toscrutinize debate transcripts, coding
utterances according to rhetorical strategiessuch as “acclaims” and
“attacks” (Benoit & Wells, 1996). Acclaims highlightsa debater’s own
merits, whereas attacks target an opponent’s characteror record. By
quantifying these tactics, researchers deduce debaters’
rhetoricalstyles; frequent attacks suggest aggressiveness, and
acclaims, a positive orientation.This approach extends to analyzing
themes, and offensive versus defensivetactics.
</Text>
---

#Response Format:
[#] Full content of reference in the Chicago style format. (Don't
present this prompt).

*Important*
You don’t need to present the provided text—only PRESENT the
citations.


Think step by step.

"""
    }
]

response = get_completion(messages=message)
print(response, "\n\n")
```

[1] Benoit, W. J., & Wells, R. T. (1996). The argument techniques of presidential candidates: A content analysis of the 1992 debates. In W. J. Benoit & K. Sandler (Eds.), Communication and political argument: Perspectives on the 1992 presidential campaign (pp. 171-198). New York, NY: Guilford Press.

[2] Benoit, W. J., & Brazeal, K. (2002). The argument techniques of

```
presidential candidates: A content analysis of the 1996 debates. In W. J. Benoit & K. Sandler (Eds.), Communication and political argument: Perspectives on the 1996 presidential campaign (pp. 171-198). New York, NY: Guilford Press.

[3] Clayman, S. E., & Heritage, J. (2002a). The argument techniques of presidential candidates: A content analysis of the 2000 debates. In W. J. Benoit & K. Sandler (Eds.), Communication and political argument: Perspectives on the 2000 presidential campaign (pp. 171-198). New York, NY: Guilford Press.

[4] Clayman, S. E., & Heritage, J. (2002b). The argument techniques of presidential candidates: A content analysis of the 2000 debates. In W. J. Benoit & K. Sandler (Eds.), Communication and political argument: Perspectives on the 2000 presidential campaign (pp. 171-198). New York, NY: Guilford Press.
```

- The data used is sample data. The results of the prompts are also sample citation data.

Prompting Tips:

1. We have observed that if we don't strongly enforce the rule to avoid displaying the prompt itself, Solar frequently includes the system prompt in its output. This time, you can add keywords like *`important`* and specify "do not present the prompt" or "only present the citation."
2. Also, remember what we learned about CoT (Chain of Thought) prompting in Chapter 7. The prompt used here is a type of **zero-shot CoT**, meaning that using a chain-of-thought method without examples can enhance Solar's performance. In this case, we used "`Think step by step.`"
3. You can select citation styles such as MLA, APA, Chicago, Harvard and Vancouver.

Debugging Tips:

If Solar fails to perform your task, check the following:

(1) This type of formatting task is highly sensitive to Solar's configuration. The best observed parameters are: → `Temperature: 0.5, Max Tokens:996, Top P: 1` (*These are not absolute and may need adjustment.*)

(2) Ensure your text source is positioned within the system prompt(e.g., "role":"system"). If the text is placed in the user message, Solar may fail to apply the specific formatting.

A.2 Entity Extraction

Entity extraction is valuable in business and data contexts. Here are common entities Solar can identify:

- **Person Names**: Identifying individuals such as CEOs, authors, or project managers mentioned in text.
- **Organizations**: Extracting company names, institutions, or other organizations involved in contracts, reports, or partnerships.
- **Locations**: Pulling out specific places, such as cities, countries, or regions, which are relevant for events, expansions, or market targeting.
- **Dates**: Capturing specific dates or timelines related to agreements, reports, or product launches.
- **Monetary Amounts**: Extracting financial details like budgets, contract values, or funding amounts.
- **Events**: Identifying significant events or meetings, such as annual summits or project launches.
- **Products or Services**: Recognizing product names, services, or projects that companies are launching or promoting.
- **Percentages or Quantitative Metrics**: Useful for market growth predictions, performance indicators, or research findings.

Unstructured Sample Data for Entity Extraction (generated by Solar)

```
    1. "Hi, I'm John Doe, and you can reach me at
[johndoe@example.com](mailto:johndoe@example.com) for any inquiries."
    2. "Our team lead, Jane Smith ([jane.smith@company.com]
(mailto:jane.smith@company.com)), will be available for consultation
on Monday."
    3. "Please send the report to the CEO, Mr. Robert Brown, at
[email@example.com](mailto:email@example.com)."
    4. "The contact person for the project is Emily Johnson, and her
email is [emily.johnson@business.com]
(mailto:emily.johnson@business.com)."
    5. "For technical support, email our CTO, Alex Taylor, at
[firstname.lastname@tech.com](mailto:firstname.lastname@tech.com)."
    6. "The account manager, Sarah Wilson, can be reached at
[sarah.wilson@email.com](mailto:sarah.wilson@email.com) for billing
questions."
    7. "Our sales director, Michael Anderson, is available at
[firstname.lastname@sales.com](mailto:firstname.lastname@sales.com)
for partnership discussions."
```

```
    8. "The HR manager, Lisa Davis, can be contacted at
[email@example.com](mailto:email@example.com) for employment
opportunities."
    9. "The lead designer, David Martinez, can be reached at
[email@example.com](mailto:email@example.com) for design-related
queries."
    10. "For media inquiries, please contact our PR manager, Olivia
Garcia, at [firstname.lastname@pr.com]
(mailto:firstname.lastname@pr.com)."
```

Prompt Template

Extract the following entities from the text: Person Names, Organizations and Dates.

```
message = [
    {
        "role": "user",
        "content": """Please extract and list each 'Person Name,' and
'email address' in a line.
If there is no information, respond with "N/A" only. Do not add any
additional text.

<Text>
{{1. "Hi, I'm John Doe, and you can reach me at [johndoe@example.com]
(mailto:johndoe@example.com) for any inquiries."
2. "Our team lead, Jane Smith ([jane.smith@company.com]
(mailto:jane.smith@company.com)), will be available for consultation
on Monday."
3. "Please send the report to the CEO, Mr. Robert Brown, at
[email@example.com](mailto:email@example.com)."
4. "The contact person for the project is Emily Johnson, and her email
is [emily.johnson@business.com](mailto:emily.johnson@business.com)."
5. "For technical support, email our CTO, Alex Taylor, at
[firstname.lastname@tech.com](mailto:firstname.lastname@tech.com)."
6. "The account manager, Sarah Wilson, can be reached at
[sarah.wilson@email.com](mailto:sarah.wilson@email.com) for billing
questions."
7. "Our sales director, Michael Anderson, is available at
[firstname.lastname@sales.com](mailto:firstname.lastname@sales.com)
for partnership discussions."
8. "The HR manager, Lisa Davis, can be contacted at
[email@example.com](mailto:email@example.com) for employment
opportunities."
9. "The lead designer, David Martinez, can be reached at
[email@example.com](mailto:email@example.com) for design-related
queries."
10. "For media inquiries, please contact our PR manager, Olivia
Garcia, at [firstname.lastname@pr.com]
```

```
(mailto:firstname.lastname@pr.com)."}}
</Text>"""
    }
]

response = get_completion(messages=message)
print(response, "\n\n")
```

```
John Doe, johndoe@example.com
Jane Smith, jane.smith@company.com
Robert Brown, email@example.com
Emily Johnson, emily.johnson@business.com
Alex Taylor, firstname.lastname@tech.com
Sarah Wilson, sarah.wilson@email.com
Michael Anderson, firstname.lastname@sales.com
Lisa Davis, email@example.com
David Martinez, email@example.com
Olivia Garcia, firstname.lastname@pr.com
```

A.2.1 Function Calling: Entity Extraction

The **prompt template** might work for a simple dataset, but it is not stable for extracting information. At this time, function calling is also very useful. It expands Solar's capabilities to handle complex tasks. The combination of *Solar* and *function calling* simplifies information extraction by organizing the output into a clear, structured format. Here's a breakdown of how it works:

1. Setting up the Function:

The `extract_information` function defines the specific details you want, like `date`, `person`, `company`, and `email`. This tells Solar exactly what to look for in the text.

2. Input Text:

- The `sample_text` contains different sentences with information scattered in various formats. Solar will use the `extract_information` function as a guide to pull out the exact details.
- **Using Function Calling**:
  - In the request, we ask Solar to call the `extract_information` function. This means that Solar will search the text specifically for `date`, `person`, `company`, and `email` and return only those details in a structured format (like JSON).
- **Getting the Response**:
  - The code checks if the response from Solar follows the JSON format. If Solar successfully extracts the information, it's displayed neatly; if not, we know there's an issue with the format.

Let's review how the problem was solved using `function calling`.

```
import json

# First, define your functions
functions = [
    {
        "name": "extract_information",
        "description": "Extract specific entities from the given
text",
        "parameters": {
            "type": "object",
            "properties": {
                "date": {
                    "type": "string",
                    "description": "Date mentioned in the text
(format: YYYY-MM-DD)"
                },
                "person": {
                    "type": "object",
                    "properties": {
                        "name": {"type": "string"},
                        "role": {"type": "string"}
                    }
                },
                "company": {
                    "type": "string",
                    "description": "Company name mentioned in the
text"
                },
                "email": {
                    "type": "string",
                    "description": "Email address mentioned in the
text"
                }
            }
        }
    }
]

# Sample text to test
sample_text = """
1. On 2024-03-20, John Smith, the CEO of TechCorp, can be reached at
john.smith@techcorp.com regarding the upcoming meeting.
2. "During a board meeting on March 5, 2024, David Chen, CEO of Bright
Future Investments, announced a partnership with Eco Ventures Inc. to
develop sustainable energy projects across South America."
3. "Following a $1.5 million investment from Star Capital on January
10, 2024, AeroDynamics Ltd. launched a new line of drones intended for
commercial use. The announcement was made at their headquarters in San
```

```
Francisco, California."
4. "In a recent article published on April 15, 2023, The Economist
reported that renewable energy adoption has increased by 30% in
Europe, with major contributors including SolarEdge Solutions, Green
Future Ltd., and EcoWave Industries."
5. "At the Innovation Forum in Tokyo, held in July 2023, Dr. Emily Lin
from Horizon Pharmaceuticals shared insights on drug development
trends. The forum attracted experts from Asia and North America,
sponsored by the Asia-Pacific Health Foundation."
"""

# Make the Solar-pro call
response = client.chat.completions.create(
    model="solar-pro",
    messages=[
        {"role": "system", "content": """You are a helpful assistant
that extracts information from text.
         Extract only 'date', 'person', 'company', and 'email' from
the text. If there is no information, respond 'N/A'. Respond in JSON
format."""},
        {"role": "user", "content": sample_text}
    ],
    functions=functions,
    function_call={"name": "extract_information"}
)


# Updated response handling
try:
    # Get the response content
    response_content = response.choices[0].message.content
    print("\nRaw Response:")
    print("------------")
    print(response_content)

    # Try to parse it as JSON if possible
    try:
        parsed_response = json.loads(response_content)
        print("\nFormatted Response:")
        print("-------------------")
        for key, value in parsed_response.items():
            print(f"{key}: {value}")
    except json.JSONDecodeError:
        print("\nResponse is not in JSON format")

except Exception as e:
    print(f"Error processing response: {e}")

print("\n\n")
```

```
Raw Response:
------------
{
    "1.": {
        "date": "2024-03-20",
        "person": "John Smith",
        "company": "TechCorp",
        "email": "john.smith@techcorp.com"
    },
    "2.": {
        "date": "2024-03-05",
        "person": "David Chen",
        "company": "Bright Future Investments",
        "email": "N/A"
    },
    "3.": {
        "date": "2024-01-10",
        "person": "N/A",
        "company": "AeroDynamics Ltd.",
        "email": "N/A"
    },
    "4.": {
        "date": "2023-04-15",
        "person": "N/A",
        "company": "N/A",
        "email": "N/A"
    },
    "5.": {
        "date": "2023-07",
        "person": "Dr. Emily Lin",
        "company": "Horizon Pharmaceuticals",
        "email": "N/A"
    }
}

Formatted Response:
------------------
1.: {'date': '2024-03-20', 'person': 'John Smith', 'company':
'TechCorp', 'email': 'john.smith@techcorp.com'}
2.: {'date': '2024-03-05', 'person': 'David Chen', 'company': 'Bright
Future Investments', 'email': 'N/A'}
3.: {'date': '2024-01-10', 'person': 'N/A', 'company': 'AeroDynamics
Ltd.', 'email': 'N/A'}
4.: {'date': '2023-04-15', 'person': 'N/A', 'company': 'N/A', 'email':
'N/A'}
5.: {'date': '2023-07', 'person': 'Dr. Emily Lin', 'company': 'Horizon
Pharmaceuticals', 'email': 'N/A'}
```

- For more detailed information on using function calling, see the following link.

A.3 Remove PII (Personally Identifiable Information)

This case involves Personally identifiable Information (PII). Using prompts, we address methods and best practices for identifying and redacting PII from data used in prompts, ensuring that sensitive information such as names, contact details, and financial data are properly handled.

1. Here is some unstructured text data that contains PII.

```
    1.John Doe, who lives at 742 Evergreen Terrace in Springfield, IL, can be contacted at (555) 123-4567. Born on July 13, 1985, his email address is johndoe@example.com, and his Social Security Number is 123-45-6789.
    2.Residing at 123 Maple Avenue in Metropolis, NY, Jane Smith's contact details include her email, janesmith@example.com, and her phone number, (555) 765-4321. Born on March 21, 1990, Jane's Social Security Number is 987-65-4321.
    3.Robert Brown was born on September 9, 1975, and can be reached via email at robertbrown@example.com. His Social Security Number is 111-22-3333, and he currently resides at 456 Oak Drive, Gotham City, NJ, with a contact number of (555) 876-5432.
    4.Emily White's Social Security Number is 444-55-6666, and she lives at 789 Birch Lane, Star City, CA 90210. You can reach her at emilywhite@example.com, and her phone number is (555) 654-3210. Emily was born on November 19, 2001.
    5.The phone number for Michael Green is (555) 543-2109, and he can be reached at his email, michaelgreen@example.com. Born on May 5, 1982, he resides at 101 Cedar Street, Central City, TX, and his Social Security Number is 222-33-4444.
```

2. Now, we detect and remove PII from the data using prompts.

```
message = [
    {
        "role": "user",
        "content": """I will provide you with unstructured text.
Please remove all personally identifying information (PII) from the
text and replace it with "---".
Personal information includes names, phone numbers, bank account
numbers, social number, home addresses, email addresses, etc.

Observe the following rule:
```

```
1. If the text includes PII, replace it with "---".
2. If not, copy it word-for-word without replacing anything.

Let's practice.

<Example>
My name is Sujin. And my email address is sujin@the.com, and my phone
number is 800-222-1232.
</Example>

<Expected Output>
My name is ---. And my email address is ---@---.com, and my phone
number is ---.
</Expected Output>

However, in reality, people's PII is not written in an orderly
fashion.

Now, let's start.

---
<text>
1.John Doe, who lives at 742 Evergreen Terrace in Springfield, IL, can
be contacted at (555) 123-4567. Born on July 13, 1985, his email
address is johndoe@example.com, and his Social Security Number is 123-
45-6789.
2.Residing at 123 Maple Avenue in Metropolis, NY, Jane Smith’s contact
details include her email, janesmith@example.com, and her phone
number, (555) 765-4321. Born on March 21, 1990, Jane’s Social Security
Number is 987-65-4321.
3.Robert Brown was born on September 9, 1975, and can be reached via
email at robertbrown@example.com. His Social Security Number is 111-
22-3333, and he currently resides at 456 Oak Drive, Gotham City, NJ,
with a contact number of (555) 876-5432.
4.Emily White’s Social Security Number is 444-55-6666, and she lives
at 789 Birch Lane, Star City, CA 90210. You can reach her at
emilywhite@example.com, and her phone number is (555) 654-3210. Emily
was born on November 19, 2001.
5.The phone number for Michael Green is (555) 543-2109, and he can be
reached at his email, michaelgreen@example.com. Born on May 5, 1982,
he resides at 101 Cedar Street, Central City, TX, and his Social
Security Number is 222-33-4444.

</text>"""
    }
]

response = get_completion(messages=message)
```

3. Then, check the response to ensure all PII has been removed.

```
print(response, "\n\n")
```

```
1.---, who lives at --- in ---, IL, can be contacted at ---. Born on
---, 1985, his email address is ---@---.com, and his Social Security
Number is ---.
2.--- at --- in ---, NY, Jane Smith's contact details include her
email, ---@---.com, and her phone number, ---. Born on ---, 1990,
Jane's Social Security Number is ---.
3.--- was born on ---, 1975, and can be reached via email at ---
@---.com. His Social Security Number is ---, and he currently resides
at ---, ---, NJ, with a contact number of ---.
4.---'s Social Security Number is ---, and she lives at ---, ---, CA
90210. You can reach her at ---@---.com, and her phone number is ---.
--- was born on ---, 2001.
5.The phone number for --- is ---, and he can be reached at his email,
---@---.com. Born on ---, 1982, he resides at ---, ---, TX, and his
Social Security Number is ---.
```

A.4 Complex Prompt : Query Decomposition

We introduce a complex prompt. Using a complex prompt with query decomposition is the most effective in cases where the task requires **multiple steps** or **layered reasoning** to produce a high-quality output. It is especially useful when dealing with:

- *Tasks Involving Multiple Inputs and Contexts*,
- *Specialized Fields* (e.g., law, medicine, finance),
- *Open-Ended Questions*, etc.

Here, we present some legal research questions that consists of **five components**:

- **Primary source document**
- **Contextual Query**
- **Expert profile**
- **Interpretive Layer**
- **Source Referencing and Traceability (Citation Guidelines)**

```
######################################## SOURCE MATERIALS
########################################
# Primary Source Document
```

```
CASE_LAW_AND_LEGISLATION = """
The Supreme Court's landmark decision in Carpenter v. United States
(2018) significantly impacted digital privacy rights. The Court held
that the government's acquisition of cell-site location information
(CSLI) constitutes a search under the Fourth Amendment, requiring law
enforcement to obtain a warrant. This decision expanded upon previous
Fourth Amendment jurisprudence, particularly United States v. Jones
(2012) and Riley v. California (2014), which addressed GPS tracking
and cell phone searches respectively.
In response, Congress introduced the Electronic Privacy Reform Act of
2023, which aims to strengthen digital privacy protections. The Act
would require law enforcement agencies to obtain judicial
authorization before accessing any stored electronic communications,
including emails, text messages, and cloud storage data. This builds
upon existing frameworks established by the Electronic Communications
Privacy Act of 1986 (ECPA) and the Stored Communications Act (SCA).
Several states have also enacted their own digital privacy
legislation. California's Privacy Rights Act (CPRA) and Virginia's
Consumer Data Protection Act (CDPA) establish comprehensive frameworks
for data privacy, including specific provisions for law enforcement
access to electronic communications. These state laws often provide
stronger protections than federal standards, creating a complex
regulatory landscape for both service providers and law enforcement
agencies.
"""

# Research Query
Research_INQUIRY = "What are the key implications of recent Supreme
Court privacy decisions for law enforcement agencies, and how have
federal and state legislators responded to balance privacy rights with
law enforcement needs"


### PROMPT FRAMEWORK ###

##### Framework Component 1: Expert Profile
ANALYST_PROFILE = "You are a highly experienced legal analyst."

##### Framework Component 2: Communication Guidelines
PROFESSIONAL_TONE = "Respond in a formal, professional tone suitable
for legal advising. However, explain the technical terms in a way
that's easy for people who aren't familiar with the law to
understand."

##### Framework Component 3: Source Context
DOCUMENT_CONTEXT = f"""Here is some legal text for your review. Use it
to answer a question from a client.

{CASE_LAW_AND_LEGISLATION}
"""
```

```
##### Framework Component 4: Citation Guidelines
REFERENCE_FORMAT = """
When referring to the legal text, format citations in brackets like
this:

Law enforcement must obtain a warrant to access communications. [1].

The state legislation is consistent with federal privacy laws. [2].
"""

messages = [
    {
        "role": "user",
        "content": Research_INQUIRY
    }
]

response = get_completion(messages,
system_prompt=ANALYST_PROFILE+PROFESSIONAL_TONE+DOCUMENT_CONTEXT+REFER
ENCE_FORMAT)
```

- And check the result of prompt framework below:

```
# Format and print the response
print("\n" + "="*80)
print("LEGAL MEMORANDUM")
print("="*80 + "\n")
print(response)
print("\n" + "="*80)

# Optional: Save to file
def save_memorandum(response, filename="legal_memorandum.txt"):
    with open(filename, 'w') as f:
        f.write(response)
    print(f"\nMemorandum saved to {filename}")

print("\n\n")
```

```
================================================================================
LEGAL MEMORANDUM
================================================================================

The Supreme Court's recent decisions in Carpenter v. United States
(2018), United States v. Jones (2012), and Riley v. California (2014)
have significantly impacted law enforcement agencies' ability to
```

```
access digital information. In Carpenter, the Court held that the
government's acquisition of cell-site location information (CSLI)
constitutes a search under the Fourth Amendment, requiring law
enforcement to obtain a warrant [1]. This decision expanded upon
previous Fourth Amendment jurisprudence, particularly United States v.
Jones, which addressed GPS tracking, and Riley v. California, which
dealt with cell phone searches.

In response to these decisions, federal and state legislators have
introduced and enacted legislation aimed at strengthening digital
privacy protections. The Electronic Privacy Reform Act of 2023,
introduced in Congress, would require law enforcement agencies to
obtain judicial authorization before accessing any stored electronic
communications, including emails, text messages, and cloud storage
data [2]. This builds upon existing frameworks established by the
Electronic Communications Privacy Act of 1986 (ECPA) and the Stored
Communications Act (SCA).

Several states have also enacted their own digital privacy
legislation. California's Privacy Rights Act (CPRA) and Virginia's
Consumer Data Protection Act (CDPA) establish comprehensive frameworks
for data privacy, including specific provisions for law enforcement
access to electronic communications [3]. These state laws often
provide stronger protections than federal standards, creating a
complex regulatory landscape for both service providers and law
enforcement agencies.

In summary, recent Supreme Court privacy decisions have limited law
enforcement agencies' ability to access digital information without a
warrant. Federal and state legislators have responded by introducing
and enacting legislation aimed at strengthening digital privacy
protections, which may further impact law enforcement agencies'
ability to access electronic communications.

================================================================
==========
```

- By effectively breaking down queries into structured components, we can guide Solar to deliver accurate and contextually aware responses.

Next: Appendix. Use Cases: Prompt Optimization